LLMs, Decoding, and Prompting APIs

Sourangshu Bhattacharya
Associate Professor
CSE, IIT Kharagpur





What We'll Cover

- Large Language Models
  - Model Architectures and training
  - Basic prompting
  - Decoding and Generation

- Prompting and APIs
  - OpenAI and Anthropic API
  - Basic Function Calling

Begin with a question

- Which of these transformer components if responsible for the “Generation” in GenAI?
  - Positional Encoding.
  - Tokenizer
  - Encoder
  - Decoder



Large Language Models

Model Architectures

BERT: Contextual LM Training

- Two self-supervised objectives - masked language modeling (MLM) and next-sentence prediction.
- A small task-specific head plus a few epochs of fine-tuning produced new state-of-the-art results across GLUE, SQuAD, and NER.
- Establishing the pretrain-then-finetune paradigm.

KEY RESULT · GLUE TEST SERVER

GLUE benchmark average (Oct 2018)



Devlin et al. (2019), Table 1. Higher is better (max 100).

Architecture

- Transformer encoder only
- BERT-Base: 110M params · 12 layers
- BERT-Large: 340M params · 24 layers
- WordPiece tokenization

Training objectives

- Masked LM — predict 15% of tokens
- Next Sentence Prediction
- BookCorpus + English Wikipedia
- ~3.3B tokens

Why it mattered

- Bidirectional context in one pass
- Transfer via fine-tuning, not features
- New SOTA on 11 NLP benchmarks
- Template for RoBERTa, ALBERT, DeBERTa

Devlin, Chang, Lee, Toutanova (2018/2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. arXiv:1810.04805.
Builds on: Vaswani et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762.

GLUE: General Language Understanding Evaluation

- Introduced in 2018, GLUE is a collection of 9 diverse NLU tasks designed to test a model's ability to understand language at multiple levels of abstraction.
- Tasks span sentiment analysis, paraphrase detection, and natural language inference, reported as a single average score.
- BERT's release immediately pushed the leaderboard above prior approaches, cementing pretrain-then-finetune as the dominant NLP paradigm.

KEY RESULT · GLUE TASK SUITE

BERT-Large on GLUE subtasks (2018)



Wang et al. (2018). GLUE: A Multi-Task Benchmark. EMNLP. Scores from Devlin et al. (2019).

Single-Sentence Tasks

- CoLA — Linguistic acceptability
- SST-2 — Binary sentiment (movie reviews)
- Metric: Matthew's corr. (CoLA), accuracy (SST-2)

Similarity & Paraphrase

- MRPC — Microsoft paraphrase pairs
- STS-B — Semantic textual similarity
- QQP — Quora duplicate questions

Inference Tasks

- MNLI — Multi-genre NLI (entailment)
- QNLI, RTE, WNLI — Textual entailment
- BERT-Large averaged 82.1 — new SOTA

Wang et al. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. EMNLP 2018. arXiv:1804.07461.
Results from: Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.

GPT-1 and GPT-2: Decoder-only Generation

OpenAI trained a causal decoder-only Transformer-based model to predict the next token.

GPT-1 (2018) showed a single generative pre-trained model could match task-specific architectures after fine-tuning.

GPT-2 (2019) scaled this to 1.5B parameters on 40 GB of web text.

KEY RESULT · ZERO-SHOT SCALING

LAMBADA accuracy — zero-shot



Radford et al. (2019), Table 3. Last word prediction on long passages.

GPT2 demonstrated usable translation, summarization, and question answering purely from natural-language prompts, without any task-specific training – zero shot.

GPT-1 (June 2018)

- Decoder-only Transformer · 117M params
- Pre-train on BookCorpus, then fine-tune
- Auxiliary LM loss stabilizes transfer
- Outperformed task-specific SOTA on 9/12 tasks

GPT-2 (February 2019)

- 1.5B parameters · WebText (40 GB)
- Zero-shot task performance from prompts
- Staged release due to misuse concerns
- First widely-felt hint of emergent capability

Radford, Narasimhan, Salimans, Sutskever (2018). Improving Language Understanding by Generative Pre-Training. OpenAI Technical Report.
Radford, Wu, Child, Luan, Amodei, Sutskever (2019). Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI Technical Report.

LAMBADA: Measuring Long-Range Text Understanding

LAMBADA (Language Modeling Broadened to Account for Discourse Aspects) tests a model's ability to predict the last word of a narrative passage where local context alone is insufficient — the entire discourse is required.

GPT-2 (1.5B) surpassed the prior state of the art in zero-shot mode — no task-specific training — across multiple long-range cloze benchmarks.

KEY RESULT · ZERO-SHOT BENCHMARKS

GPT-2 1.5B zero-shot accuracy (2019)



Radford et al. (2019). GPT-2, Table 4. Zero-shot on each benchmark.

LAMBADA Benchmark

- Released 2016; 10K cloze-style test passages
- Last word unpredictable from final sentence alone
- Drawn from BookCrossing fiction dataset

Why It's Hard

- Requires reading 4+ sentence narrative context
- N-gram models score near 0% accuracy
- Tests genuine discourse comprehension, not co-occurrence

GPT-2's Significance

- First model to surpass prior LAMBADA SOTA zero-shot
- Scale alone (without task supervision) drives gains
- Early evidence that LLMs encode discourse structure

Paperno et al. (2016). The LAMBADA dataset: Word prediction requiring a broad discourse context. ACL 2016. arXiv:1606.06031.
Radford, Wu, Child, Luan, Amodei, Sutskever (2019). Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI Technical Report.

T5 and BART: Text-to-Text and Denoising Seq2Seq

Google's T5 cast every NLP task — classification, translation, summarization, question answering — as string-to-string prediction with a single model and loss. Facebook AI's BART combined a bidirectional encoder with an autoregressive decoder, pre-training on varied denoising corruptions. Together they established seq2seq pre-training as a strong alternative to BERT-style encoders.

KEY RESULT · SUPERGLUE TEST SET

Closing the gap to human performance



Raffel et al. (2020), Table 14. SuperGLUE average score.

T5 (Raffel et al., 2019)

- Encoder–decoder, span-corruption objective
- Trained on C4 (750 GB cleaned web text)
- Scales from 60M to 11B parameters
- Unifies all tasks as text → text

BART (Lewis et al., 2019)

- BERT-like encoder + GPT-like decoder
- Denoising objective with varied corruptions
- Strong on summarization and generation
- Template for later seq2seq systems (mBART, Pegasus)

Raffel et al. (2019/2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5). JMLR. arXiv:1910.10683.
Lewis et al. (2019/2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ACL. arXiv:1910.13461.

GPT-3: Few-Shot Learning and the Prompting Paradigm

GPT-3 scaled the GPT-2 recipe by two orders of magnitude.

A sufficiently large language model can perform novel tasks from a handful of demonstrations in the prompt — no gradient updates, no fine-tuning. The era of API-served foundation models and prompt engineering began here.

175 B	~300 B	2,048	117×
PARAMETERS	TRAINING TOKENS	CONTEXT WINDOW	LARGER THAN GPT-2

TriviaQA accuracy (closed-book, GPT-3 175B)



Brown et al. (2020), Table H.9. No parameter updates — adaptation purely via the prompt.

[DOC] [TLE] Breed Breakdown: Doberman Pinscher - unleashmagazine.comBreed Breakdown: Doberman Pinscher [PAR] Breed Breakdown: Doberman Pinscher [PAR] Fast as the wind and strong as a bull, the Doberman Pinscher has proven itself to be one of the world's elite canines. They are known for their agility, intelligence, speed, stamina and strength. Amazingly enough, this breed originated in Germany and was bred by a German man named Karl Friedrich Louis Dobermann in the 1860s for those purposes. This is of course where the Doberman, also known as Dobes, get their name. Louis wanted to create a guard dog that would protect him as he traveled through dangerous communities filled with	Which actor had a Doberman Pinscher called Kirk?	["william shatner"]

Brown et al. (2020). Language Models are Few-Shot Learners. NeurIPS. arXiv:2005.14165.

GPT-3: In-Context Learning

GPT-3 (175B parameters) demonstrated that scale alone enables in-context learning — solving novel tasks from prompt examples with no gradient updates.

- **Zero-shot:** Task description only, no examples given
- **Few-shot:** 10–100 examples in-context; approaches fine-tuned models

KEY RESULT · IN-CONTEXT LEARNING EMERGES

GPT-3 175B TriviaQA accuracy (%)



Brown et al. (2020). GPT-3, Table 3.4. TriviaQA closed-book.

Zero-Shot Learning

- Only a task description in the prompt; no examples provided
- Demonstrates world knowledge captured during pre-training
- Useful baseline; no prompt engineering required

Few-Shot Learning

- 10–100 input-output examples precede the query
- No weight updates — adapts via attention over context
- Approaches fine-tuned smaller models on many tasks

Scale and Emergence

- Smaller GPT-3 variants show weaker in-context learning
- Capability emerges sharply with scale — not a smooth trend
- Hints at qualitative shifts beyond mere parameter count

Brown, Mann, Ryder et al. (2020). Language Models are Few-Shot Learners. NeurIPS 2020. arXiv:2005.14165.

InstructGPT and RLHF: Aligning Models with Human Intent

A raw language model predicts likely continuations — not helpful answers. InstructGPT aligned GPT-3 with human preferences through three stages: supervised fine-tuning on demonstrations, a reward model trained from pairwise comparisons, and policy optimization with PPO. Labelers preferred the 1.3B InstructGPT over the 175B base GPT-3. Released publicly as ChatGPT in late 2022, the technique defined what "using an LLM" would feel like.

KEY RESULT · HUMAN PREFERENCE

Win-rate vs 175B SFT baseline



Ouyang et al. (2022), Figure 1. Labeler preference on held-out prompts.

01
Supervised Fine-Tuning

- Human-written prompt/response pairs
- Fine-tune base model on demonstrations
- Teaches format and style of helpful replies

02
Reward Model

- Annotators rank model outputs
- Train a reward model on pairwise comparisons
- Proxy for human preference

03
RL with PPO

- Policy optimized against reward model
- KL penalty keeps policy near SFT
- Yields helpful, harmless, honest model

Ouyang et al. (2022). Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS. arXiv:2203.02155.
Foundational: Christiano et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS. arXiv:1706.03741. · Schulman et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

The Open-Weights Wave: LLaMA, Mistral, Falcon

Through 2022 the frontier was locked behind commercial APIs. Meta's LLaMA (Feb 2023) opened the gates with a family of models whose weights were distributed to researchers. LLaMA 2 added commercial licensing; Mistral brought compact dense and MoE variants; TII's Falcon pushed training-data transparency. A rich ecosystem of fine-tunes — Alpaca, Vicuna, and thousands more — made LLM customization accessible to any lab with a single GPU server.

KEY RESULT · OPEN MODELS CATCH UP

MMLU 5-shot accuracy (late 2023)



Jiang et al. (2024); Touvron et al. (2023). MMLU: 57-subject knowledge test.

LLaMA / LLaMA 2 / LLaMA 3

- Meta, Feb 2023 → 2024
- 7B – 70B dense models
- LLaMA 2: commercial license
- Template for modern open models

Mistral & Mixtral

- Mistral 7B (Sep 2023)
- Mixtral-8x7B — open MoE (Dec 2023)
- Strong quality-per-parameter
- Apache-2.0 licensing

Falcon (TII, UAE)

- Falcon-40B, Falcon-180B
- Training data (RefinedWeb) released
- Emphasis on transparency
- Later joined by Qwen, DeepSeek, Yi

Touvron et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971. · Touvron et al. (2023). Llama 2. arXiv:2307.09288.
Jiang et al. (2023). Mistral 7B. arXiv:2310.06825. · Jiang et al. (2024). Mixtral of Experts. arXiv:2401.04088. · Almazrouei et al. (2023). The Falcon Series of Open Language Models. arXiv:2311.16867.

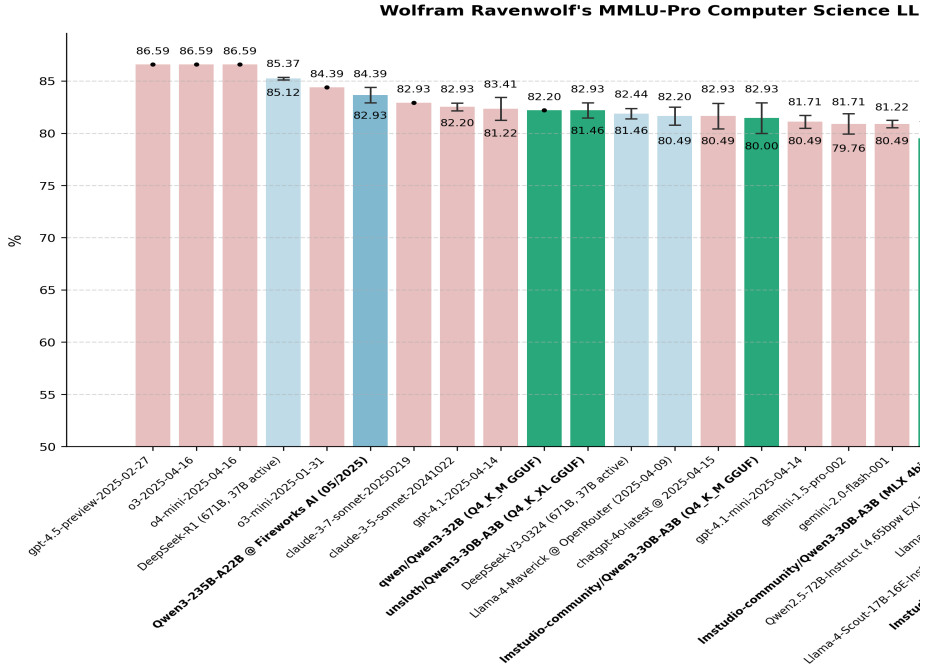
Massive Multitask Language Uunderstanding

Key Aspects of the MMLU Benchmark:

Scope: Covers 57 diverse subjects, including STEM, humanities, and social sciences, testing both knowledge and problem-solving skills.

Format: Multiple-choice questions (usually 4 options), testing both knowledge base and reasoning ability.

Purpose: It is used as a standard measure of general knowledge in AI model reports.



Touvron et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971. · Touvron et al. (2023). Llama 2. arXiv:2307.09288.
Jiang et al. (2023). Mistral 7B. arXiv:2310.06825. · Jiang et al. (2024). Mixtral of Experts. arXiv:2401.04088. · Almazrouei et al. (2023). The Falcon Series of Open Language Models. arXiv:2311.16867.

The Multimodal and Reasoning Era

The frontier bifurcated along two axes. On the input side, models became natively multimodal: GPT-4 (2023) accepted images; Gemini (2023) was trained on text, images, audio, and video end-to-end; Claude 3 and 4 offered long-context vision-language reasoning.

On the reasoning side, a new generation of systems — OpenAI's o1/o3, DeepSeek-R1, and similar — spent significant inference-time compute on chain-of-thought search, optimized via reinforcement learning on verifiable outcomes. For the first time, models routinely solved competition-level mathematics and programming problems.

Natively multimodal frontier

- GPT-4 / GPT-4V (OpenAI, Mar 2023)
- Gemini 1.0 / 1.5 / 2 (Google DeepMind)
- Claude 3 & 4 (Anthropic) · long context, vision
- Unified tokenizers over text, image, audio

Inference-time reasoning

- OpenAI o1 (Sep 2024), o3 (2025)
- DeepSeek-R1 — open reasoning via RL (2025)
- Chain-of-thought as a first-class output
- Compute shifts from training to inference

OpenAI (2023). GPT-4 Technical Report. arXiv:2303.08774. · Gemini Team (2023). Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805. · Anthropic (2024). The Claude 3 Model Family: Opus, Sonnet, Haiku. Model Card. OpenAI (2024). Learning to Reason with LLMs (o1 system card). · DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

A Map of Innovations: Architecture · Training · Data · Alignment · Inference

Era	Architecture	Training / Data	Alignment / Inference
Pre-2018 (word2vec · ELMo)	Shallow embeddings; biLSTM LM	Co-occurrence statistics; small corpora	Feature-based transfer
BERT (2018)	Deep bidirectional encoder	MLM + NSP on 3.3B tokens	Fine-tune with small task head
GPT-1 / GPT-2 (2018–19)	Decoder-only causal Transformer	BookCorpus → WebText (40 GB)	Zero-shot task transfer via prompts
T5 / BART (2019–20)	Encoder–decoder seq2seq	C4 span-corruption; denoising	Unified text-to-text interface
GPT-3 (2020)	175B dense Transformer	~300B tokens, Common Crawl mix	In-context / few-shot learning
Scaling · Chinchilla (2020–22)	Same arch, principled sizing	~20 tokens / parameter optimal	Predictable loss curves
InstructGPT · RLHF (2022)	Base LM + reward model	Human demos + preference data	SFT + RLHF produce helpful assistants
Open-weights (2023–24)	LLaMA, Mistral (MoE), Falcon	Open corpora; permissive licenses	On-device, fine-tunable, auditable
Multimodal · Reasoning (2023–25)	Vision-language; deliberative policies	Text + image + audio; RL on outcomes	Inference-time chain-of-thought search

Progress compounds along four axes — architecture, training data, alignment, and inference-time compute — not any single breakthrough.

Question

- Which of these is a major disadvantage of reasoning models?
    - They are not very good at reasoning.
    - They hallucinate more than non-reasoning models
    - They are time / token consuming.
    - None



Large Language Models

Basic Prompting

What are prompts?

- **Prompts** involve instructions and context passed to a language model to achieve a desired task
- **Prompt engineering** is the practice of developing and optimizing prompts to efficiently use language models (LMs) for a variety of applications
  - Prompt engineering is a useful skill for AI engineers and researchers to improve and efficiently use language models

```
What is prompt engineering?

Prompt engineering is a process of creating a set of
or questions, that are used to guide the user toward a
outcome. It is an effective tool for designers to
experiences that are easy to use and intuitive. This
often used in interactive design and software
it allows users to easily understand how to interact
system or
product..
```

First Basic Prompt

Instruction

```
The sky is

blue

The sky is a beautiful blue color during the day. The
blue of the sky is created by the Earth's atmosphere
scattering the sun's light. The blue is a result of the
air molecules in the atmosphere reflecting the shorter
wavelength of blue light back to our eyes.
```

Response

Model: "text-davinci-003"
temperature: 0.7
top-p: 1

I ran ->

```python
from openai import OpenAI
client = OpenAI()  # reads OPENAI_API_KEY

response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[
    {"role": "user",
     "content": "The sky is"}
  ],
  temperature=0.7,
  max_tokens=200
)
print(response.choices[0].message.content)
```

Output ->

```
(LLMs) sourangshu@Sourangshus-MacBook-Air-3 openAI % python test-openAPI.py
The sky is a vast expanse above us, usually appearing blue during the day due to
 the scattering of sunlight, and it can transform into a stunning display of col
ors at sunrise and sunset. At night, it reveals a tapestry of stars, planets, an
d sometimes the moon, offering a glimpse into the universe beyond our planet. Th
e sky can also be home to various weather phenomena, from fluffy white clouds to
 stormy gray skies. What aspect of the sky are you interested in?
```

A more advanced Prompt

```
  messages=[
    {"role": "system",
     "content": """You are a Socratic Debugger. Never fix code directly — guide the
user to find the bug themselves through questions and analogies.

Rules:
1. Don't reveal the bug until the user says "give up" or asks 3+ times.
2. Ask what they *think* the code does vs. what it actually does.
3. Respond only with probing questions or nudging analogies.
4. End each reply with [Proximity: X/10] — how close they are.
5. At proximity 9+, give one direct hint. Celebrate when they find it!"""},

    {"role": "user",
     "content": """My Python function returns wrong averages — why?

def average(nums):
    total = 0
    for n in nums:
        total =+ n
    return total / len(nums)

print(average([10, 20, 30]))  # Expected: 20.0, Got: 30.0"""}
  ],
```

<- Prompt

Output

|

V

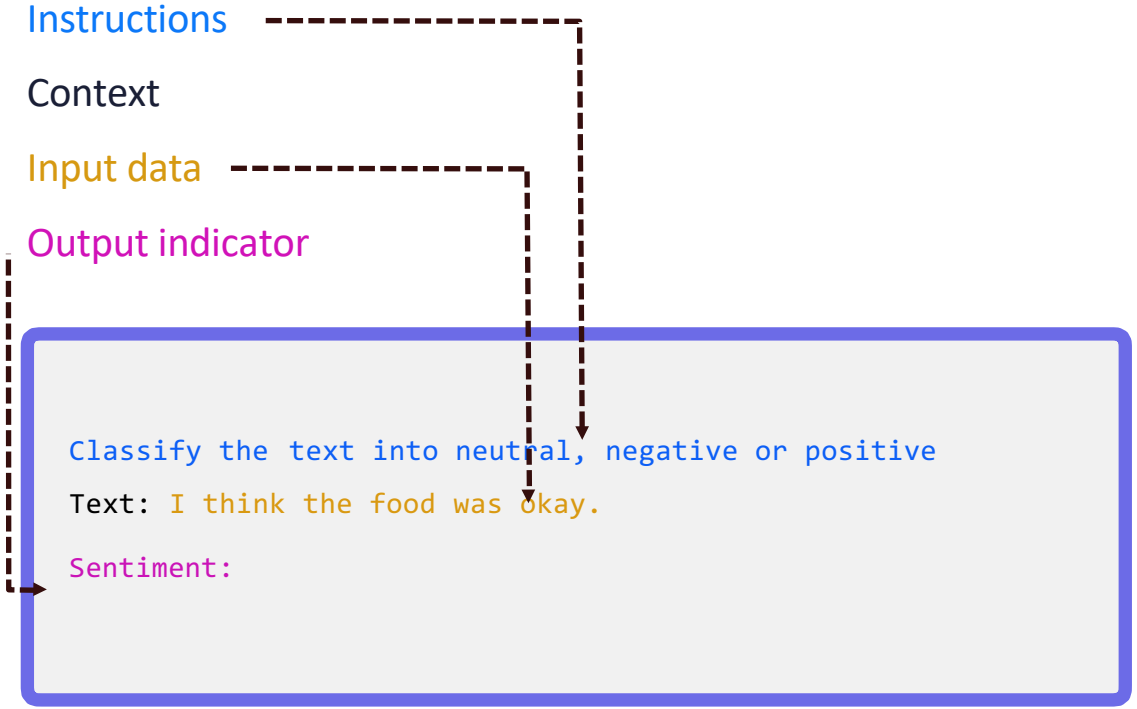
```
What do you think the line `total =+ n` is doing in your function?
 Does it look like it's adding `n` to `total` as you might expect?


Is there a difference between the operation you intended and what
is actually happening in that line?

[Proximity: 4/10]
```

Elements of a Prompt

- A prompt is composed with the following components:
  - Instructions
  - Context
  - Input data
  - Output indicator



Large Language Models

Response Generation and Decoding

Which words do we generate at each step?

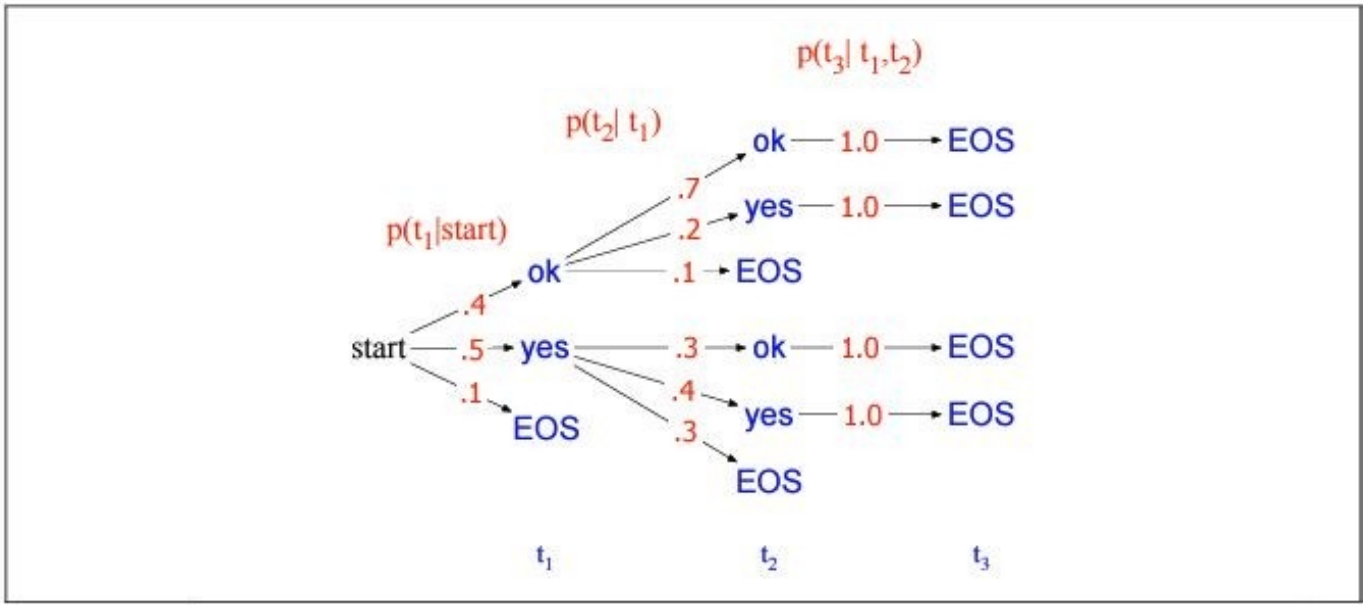
Greedy Decoding

One simple way is to always generate the most likely word given the context

Generating the most likely word given the context is called *greedy decoding*.

$$\hat{w}_t = \arg\max_{w \in V} P(w|w_{<t})$$

What is the issue with greedy decoding?



- *What will greedy decoding choose?*
- *What is globally optimal?*

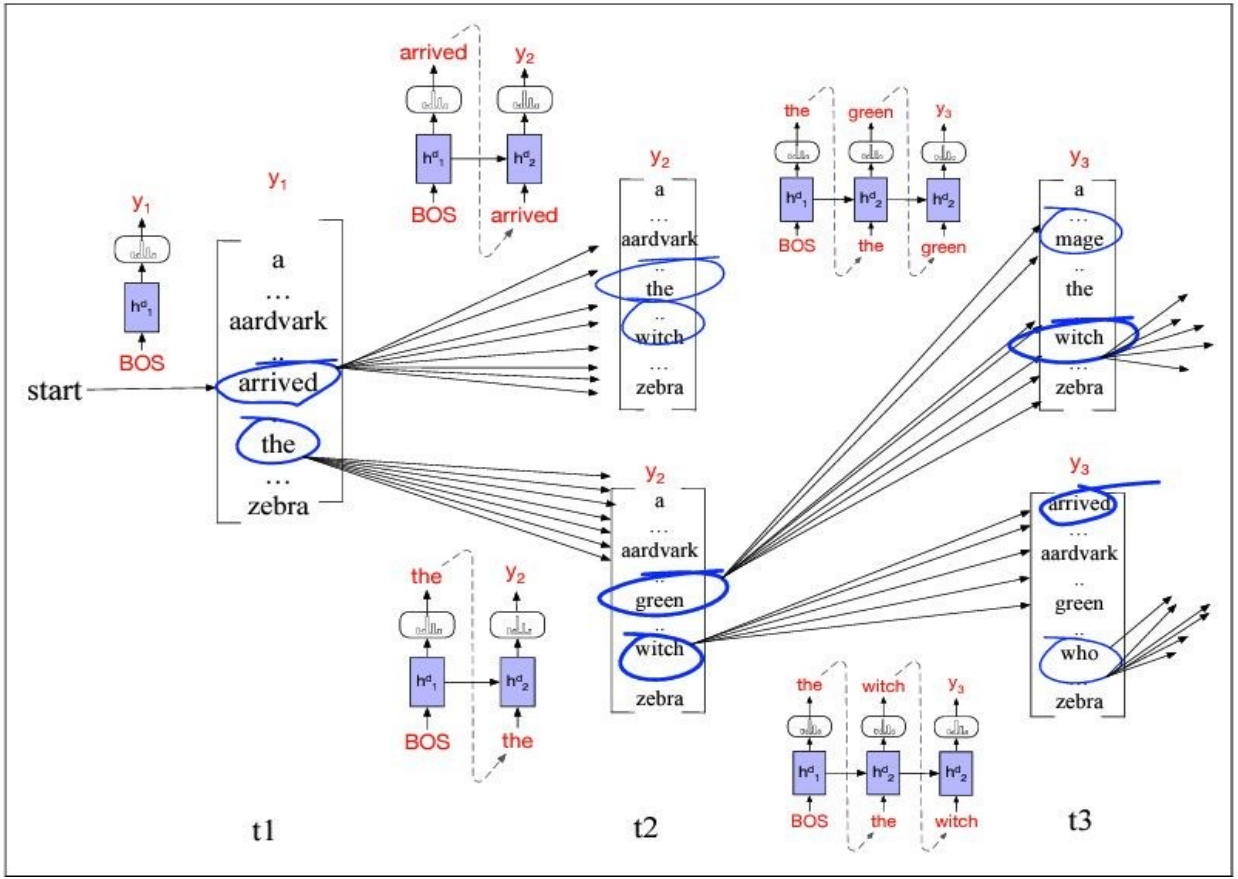
Source: https://web.stanford.edu/~jurafsky/slp3.

Beam Search

Core Idea

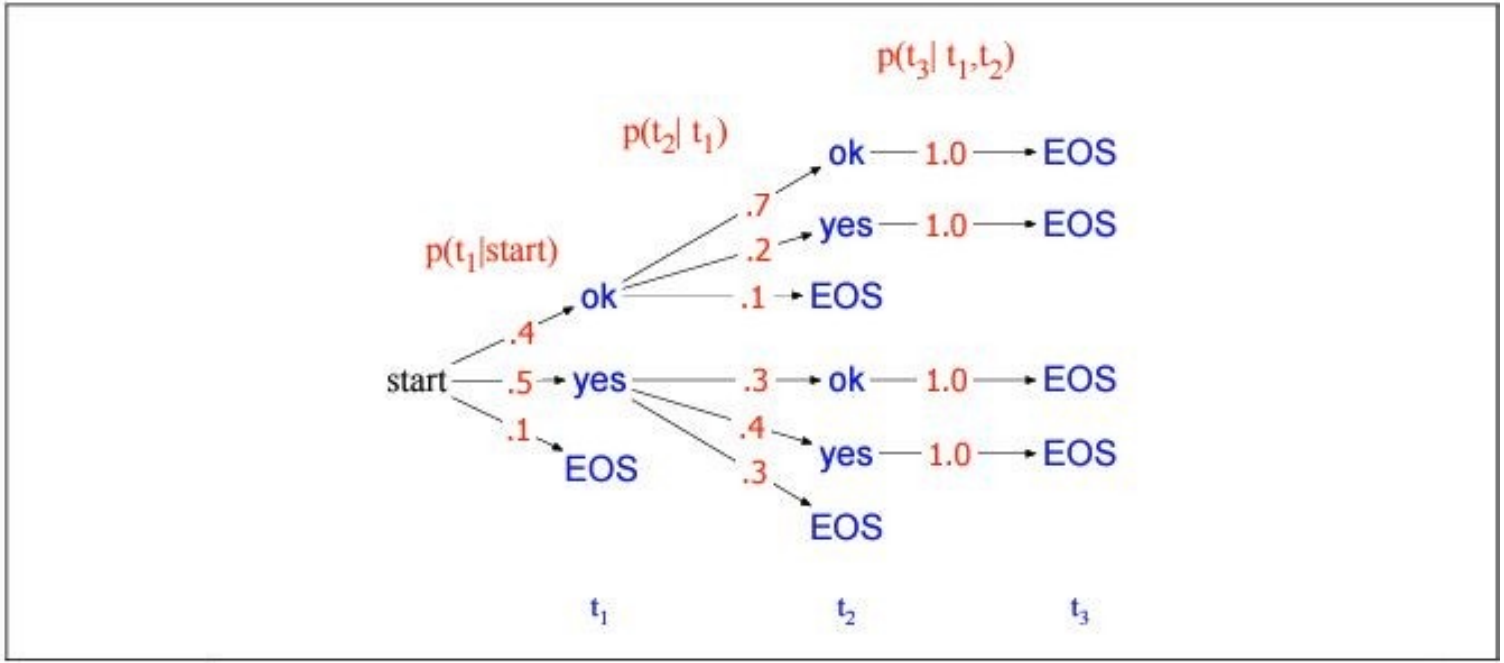
- On each step of the decoder, keep track of the k most probable partial translations (hypotheses)
- k is the beam size / beam width (in practice around 5 to 10)

Beam Search: How does it work?



https://web.stanford.edu/~jurafsky/slp

Beam Search on the prev. example



Take Beam Size = 2. Is the Beam search solution same as the optimal solution?

https://web.stanford.edu/~jurafsky/slp3

Greedy and Beam Search are deterministic!

- Greedy decoding as well as Beam Search decoding will give a "deterministic" output
- Other common decoding algorithms involve "sampling", and bring in some degree of "randomness"

$x \sim p(x) \rightarrow$ choose x by sampling from the distribution $p(x)$

Random Sampling

i $\leftarrow$ 1
$w_i \sim$ p(w)
while w_i != EOS
  i $\leftarrow$ i + 1
  $w_i \sim$ p($w_i \mid w_{<i}$)

Quality vs Diversity trade-off

Various sampling methods enable trading off two important factors in generation: *quality* and *diversity*.

Quality vs Diversity trade-off

- Methods that emphasize the most probable words tend to produce more coherent and accurate generations but also tend to be repetitive and boring
- Methods that give bit more weight to the middle probability words tend to be more creative and diverse, but likely to be incoherent and less factual

Random Sampling with Temperature

Intuition from thermodynamics

A system at a *high temperature* is flexible and can explore various states, while a system at a *low temperature* is likely to explore a subset of lower energy (better) states

How is this implemented

Divide the logits by a temperature parameter $\tau \in (0, 1]$ before passing it through softmax

Random sampling: $y = softmax(u)$

Random sampling with temperature: $y = softmax(u/\tau)$

Temperature

Definition: Temperature scales the logits before softmax. P(token) = softmax(logits / T)

T = 0.3 Very Cold

Almost deterministic.
Always picks the most likely token.

Example output:

"Paris is the capital of France."

T = 1.0 Default

Natural distribution.
Balanced creativity + coherence.

Example output:

"Paris is a stunning European capital."

T = 1.8 Very Hot

High randomness.
Creative but can be incoherent.

Example output:

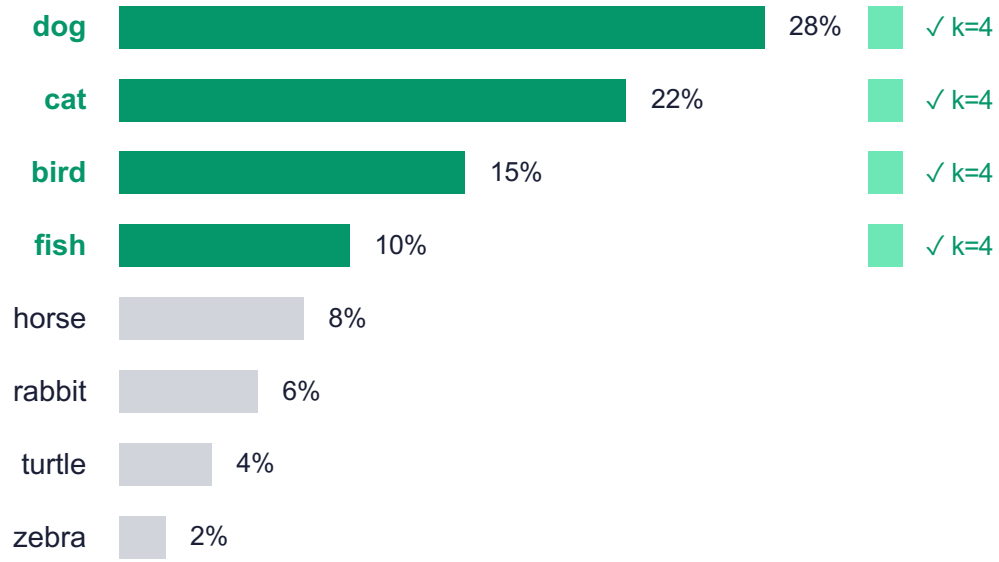
"Paris is cheese-dream cobblestone forever!"

Tip: Use T≈0.7 for factual tasks, T≈1.2 for creative writing, T>1.5 only for brainstorming.

Top-k Sampling

Keep only the k most probable next tokens; redistribute probability ma

Vocabulary Probabilities (top 8 tokens):



How It Works (k = 4)

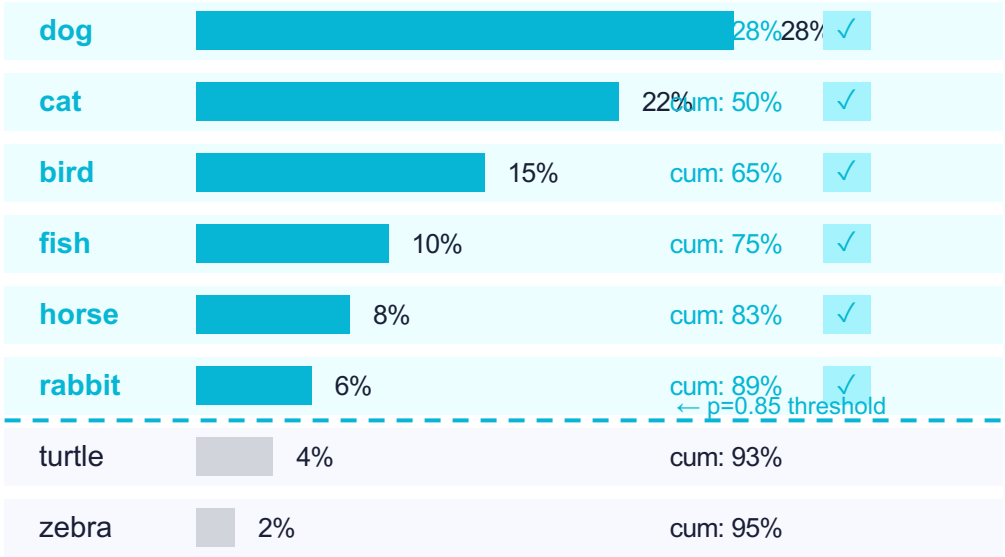
1. Compute logits over full vocabulary (e.g. 50,000 tokens)
2. Sort tokens by probability (descending)
3. Keep only top-k tokens. Discard the rest.
4. Re-normalize probabilities to sum to 1
5. Sample from the reduced distribution

Low k = conservative, High k = diverse

Top-p (Nucleus) Sampling

Sample from the smallest set of tokens whose cumulative probability ≥ p. Dynamic vocabulary cutoff!

p = 0.85 example (cumulative probability reaches 0.85):



Top-p vs Top-k

Aspect	Top-k	Top-p
Cutoff type	Fixed count (k tokens)	Dynamic (by prob mass)
Low confidence	Always uses k tokens	Restricts to fewer
High confidence	May include garbage	Stays focused
Typical value	k = 40–50	p = 0.9–0.95

Repetition Penalty & Frequency Penalty

Repetition Penalty

repetition_penalty = 1.3

Divides logit of any token that already appeared in the context by the penalty value.

```
logit[t] /= penalty  (if token t seen before)
```

❌ **Without penalty:**

The cat sat. The cat sat. The cat sat. The cat...

✅ **With penalty:**

The cat sat on the mat and then yawned slowly.

Values 1.0 = no effect. 1.2–1.5 is typical. Too high → avoids even necessary repeats.

Frequency Penalty

frequency_penalty = 0.5

Penalizes tokens proportional to how many times they have appeared so far (OpenAI style).

```
logit[t] -= freq_penalty × count(t)
```

❌ **Without penalty:**

Paris Paris Paris is Paris the capital Paris Paris

✅ **With penalty:**

Paris is the capital of France and a major hub.

Range: 0.0 to 2.0. Higher = more penalty. Presence penalty is a binary version (0 or 1).

Max Tokens & Stop Sequences

Max Tokens

Controls the maximum number of tokens the model will generate in one response.

max_tokens=20

"The transformer architecture was introduced in 2017 by Google researchers in the..."

max_tokens=50

"...paper Attention Is All You Need. It replaced RNNs with self-attention mechanisms, enabling much faster training."

⚠️ *Tokens ≠ Words. On average, 1 word ≈ 1.3 tokens. "ChatGPT" = 3 tokens.*

Stop Sequences

Custom strings that signal the model to stop generating immediately. Useful for structured output.

Code generation

stop: ["```", "def "]

Stop after code block ends

Q&A system

stop: ["\nUser:"]

Stop before next turn

JSON output

stop: ["}\n"]

Stop after JSON closes

💡 *Combine with max_tokens for full output control.*

Parameter Cheat Sheet

Parameter	Range	Default	Creative	Factual	Description
temperature	0.0 – 2.0	1.0	1.2	0.3	Scales randomness of output distribution
top_k	1 – 1000+	50	80	10	Keeps only k highest-probability tokens
top_p	0.0 – 1.0	1.0	0.95	0.7	Nucleus sampling: cumulative prob cutoff
repetition_penalty	1.0 – 2.0	1.0	1.2	1.1	Reduces logits for already-seen tokens
frequency_penalty	0.0 – 2.0	0.0	0.4	0.1	Proportional penalty based on token count
max_tokens	1 – 128k+	Model	1024	256	Hard cap on generated token count

⚠ *Never use top_k and top_p together at extreme values — they compound and may produce degenerate output. Use one or the other, then tune temperature.*

Question

- Which of the following is most effective in reducing hallucinations?
  - Temperature
  - Top-k sampling
  - Top-p Sampling
  - Max-Tokens



Tutorial: Putting It All Together

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
  model="claude-sonnet-4-20250514",
  max_tokens=512,       # ← output cap
  messages=[{"role":"user",
    "content":"Write a poem about AI"}],
)

# OpenAI-style params (via API)
# temperature  = 1.1  # creative
# top_p        = 0.9  # nucleus
# top_k        = 50   # top-k
# stop         = ['\n\n']

print(response.content[0].text)
```

Use-Case Presets

📖 Factual Q&A

temp=0.3, top_p=0.7
max_tokens=256

✍️ Creative Writing

temp=1.2, top_p=0.95
rep_penalty=1.2

💻 Code Generation

temp=0.2, top_k=20
stop=["\n```"]

🧠 Brainstorming

temp=1.5, top_k=80
max_tokens=1024

Prompting and APIs

OpenAI API

OpenAI API (Introduction)

Overview:

- Proprietary API · pay-per-token
- Industry-standard LLM platform
- Supports text · code · image · audio

Core Capabilities:

- Chat completions (LLM interaction)
- Function / tool calling
- Structured outputs (JSON mode)
- Embeddings · retrieval

Modalities Supported:

- Text generation
- Code generation
- Image generation & vision
- Audio (speech-to-text, text-to-speech)

Key Advantages:

- High reliability · production-ready
- Scalable infrastructure
- Wide model ecosystem

Frontier Models

These are OpenAI's most advanced general-purpose models, designed for high performance across reasoning, coding, and agentic workflows.

GPT-5.4

- Most advanced model for **intelligence at scale**
- Strong in **agentic workflows and complex reasoning**
- High accuracy for professional and enterprise use cases
- Handles long, multi-step tasks efficiently

GPT-5.4 mini

- Cost-efficient version of GPT-5.4
- Optimized for **high-volume and low-latency tasks**
- Good balance between performance and cost
- Suitable for scalable applications



Basic Chat Completion (OpenAI API)

Basic Chat Completion

```
from openai import OpenAI
client = OpenAI()  # reads OPENAI_API_KEY

response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[
    {"role": "system",
     "content": "You are a helpful assistant."},
    {"role": "user",
     "content": "Explain RAG in one sentence."}
  ],
  temperature=0.7,
  max_tokens=200
)
print(response.choices[0].message.content)
```

Step 1: User sends messages to API

Step 2: Model processes system + user input

Step 3: Model generates response

Step 4: API returns structured output

Roles in Prompting (OpenAI API)

Core Message Roles for (Chat Completions):

- system: Sets the overall behavior, tone, and constraints for the assistant (e.g., "You are a helpful assistant").
- developer (New): Designed for developers to provide specialized instructions, giving more control over API interaction than the traditional system role.
- user: Represents the input from the human user, including questions or instructions.
- assistant: Represents the previous responses generated by the AI, helping the model maintain context in a conversation.

Roles in Prompting (OpenAI API)

The assistant role is used in the messages array to represent the model's past or simulated responses. It can be use for:

- Maintaining Conversation History

```
"messages": [
  {"role": "developer", "content": "You are a helpful travel guide."},
  {"role": "user", "content": "Tell me about Paris."},
  {"role": "assistant", "content": "Paris is the capital of France, known
  {"role": "user", "content": "What is the best time to visit?"}
]
```

- Few-shot prompting:

```
"messages": [
  {"role": "developer", "content": "You translate text into emojis."},
  {"role": "user", "content": "I love coffee."},
  {"role": "assistant", "content": "❤️ ☕"},
  {"role": "user", "content": "The weather is sunny."},
  {"role": "assistant", "content": "☀️ ☁️"},
  {"role": "user", "content": "I am going for a run."}
]
```

Roles in Prompting (OpenAI API)

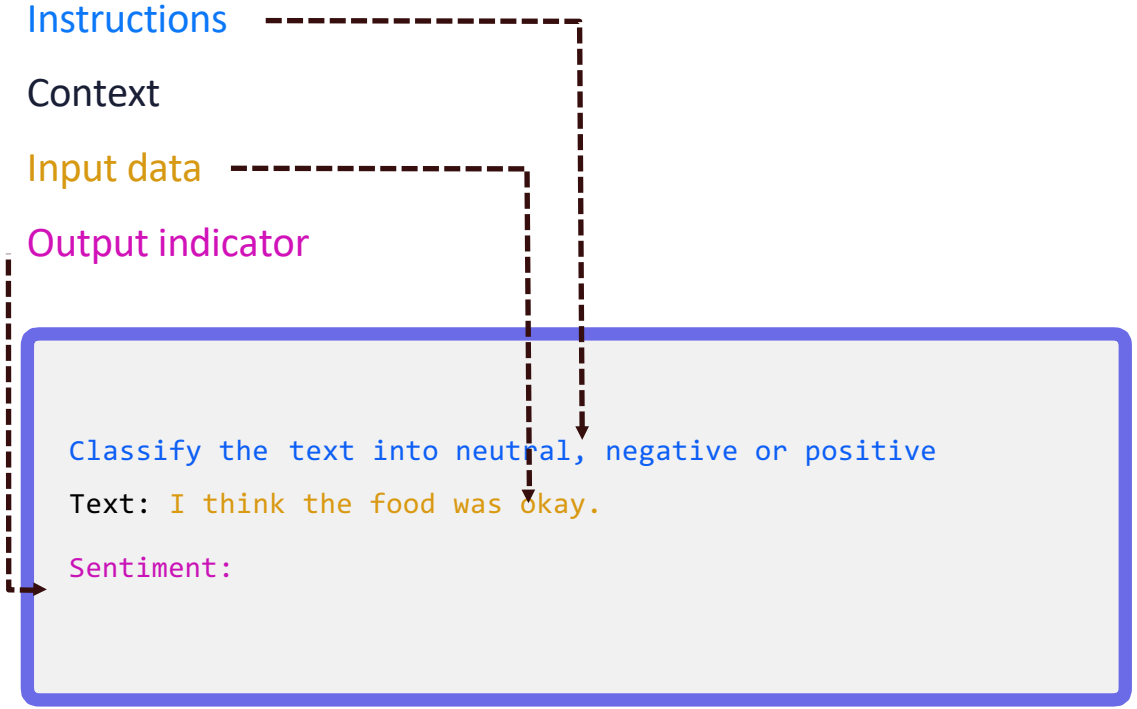
What actually happens under the hood

Before reaching the model, the structured input is converted into a **single token sequence**, something conceptually like:

```
<|system|>
You are helpful
<|user|>
Hello
<|assistant|>
Hi!
```

Elements of a Prompt

- A prompt is composed with the following components:
  - Instructions
  - Context
  - Input data
  - Output indicator



Tool Calling

Model can call external functions via structured outputs

Converts natural language → function arguments

Flow:

- User query → Model → Tool selection → Function call → Response

Key Features:

- JSON schema-based inputs
- Automatic tool selection
- Reliable structured outputs
- Integrates APIs · databases · services

Tool Calling Example – LCM calculation

How to compute LCM (lowest common multiple) of two given integers?

Input: “What is the LCM of 84 and 120?”

total_tokens=425

```python
from openai import OpenAI
client = OpenAI()  # reads OPENAI_API_KEY

response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[
    {"role": "user",
     "content": "You are a helpful assistant"},
    {"role": "user",
     "content": "What is the LCM of 84 and 120?"}
  ],
  temperature=0.7,
  max_tokens=1000
)
print(response.choices[0].message.content)
```

```
1. **Find the prime factorization of each number**:
   - For 84:
     - \( 84 = 2^2 \times 3^1 \times 7^1 \)
   - For 120:
     - \( 120 = 2^3 \times 3^1 \times 5^1 \)

2. **Identify the highest power of each prime factor present in both factorizations**:
   - For the prime factor 2: the highest power is \( 2^3 \) (from 120).
   - For the prime factor 3: the highest power is \( 3^1 \) (common in both).
   - For the prime factor 5: the highest power is \( 5^1 \) (from 120).
   - For the prime factor 7: the highest power is \( 7^1 \) (from 84).

3. **Multiply these together to find the LCM**:
   \[
   \text{LCM} = 2^3 \times 3^1 \times 5^1 \times 7^1
   \]
   Calculating this step-by-step:
   - \( 2^3 = 8 \)
   - \( 3^1 = 3 \)
   - \( 5^1 = 5 \)
   - \( 7^1 = 7 \)

   Now multiply these values:
   \[
   8 \times 3 = 24
   \]
   \[
   24 \times 5 = 120
   \]
   \[
   120 \times 7 = 840
   \]

Thus, the LCM of 84 and 120 is **840**.
```

Tool Calling Example – LCM calculation

: “What is the LCM of 84 and 120?”

```python
from openai import OpenAI
import json
import math
client = OpenAI()
# 1. Define a list of callable tools for the model
tools = [
    {
        "type": "function",
        "name": "compute_lcm",
        "description": "Computes the Least Common Multiple (LCM) of two integers.",
        "parameters": {
            "type": "object",
            "properties": {
                "a": {"type": "integer", "description": "The first integer."},
                "b": {"type": "integer", "description": "The second integer."}
            },
            "required": ["a", "b"],
            "additionalProperties": False
        }
    }
]
# Tool implementation
def compute_lcm(a: int, b: int) -> int:
    return abs(a * b) // math.gcd(a, b)
```

Setup the function

Tool Calling Example – LCM calculation

```python
# Create a running input list we will add to over time
input_list = [
    {"role": "user", "content": "What is the LCM of 84 and 120?"}
]
# 2. Prompt the model with tools defined
response = client.responses.create(
    model="gpt-4o-mini",
    tools=tools,
    input=input_list,
)
# Save function call outputs for subsequent requests
input_list += response.output
for item in response.output:
    if item.type == "function_call":
        if item.name == "compute_lcm":
            # 3. Execute the function logic for compute_lcm
            args = json.loads(item.arguments)
            lcm = compute_lcm(args["a"], args["b"])

            # 4. Provide function call results to the model
            input_list.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": f"LCM: {lcm}"
            })

print("Final input:")
print(input_list)
```

Create the first set of function calls Using an LLM call

Execute the function

Create the input for the second LLM Call using the output from the Function call

Tool Calling Example – LCM calculation

```
response = client.responses.create(
    model="gpt-4o-mini",
    instructions="Respond only with the LCM of the two integers provided.",
    tools=tools,
    input=input_list,
)

# 5. The model should be able to give a response!
print("Final output:")
print(response.model_dump_json(indent=2))
print("\n" + response.output_text)
```

Use the output from Function execution to Ask LLM for the final answer.

Total tokens: 123

OpenAI API

GPT-4o, GPT-4.1, o3 — the industry-standard REST API

Key Features

- Proprietary · Pay-per-token
- JSON mode & Structured Outputs
- Function / tool calling
- Vision, audio, embeddings
- Fine-tuning support

Basic Chat Completion

```
from openai import OpenAI
client = OpenAI()  # reads OPENAI_API_KEY

response = client.chat.completions.create(
  model="gpt-4o",
  messages=[
    {"role": "system",
     "content": "You are a helpful assistant."},
    {"role": "user",
     "content": "Explain RAG in one sentence."}
  ],
  temperature=0.7,
  max_tokens=200
)
print(response.choices[0].message.content)
```

Function / Tool Calling

```
tools = [{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {"type": "string"}
      },
      "required": ["location"]
    }
  }
}]
resp = client.chat.completions.create(
  model="gpt-4o", messages=msgs,
  tools=tools, tool_choice="auto"
)
```

JSON mode: Set response_format={"type":"json_object"} — model always returns valid JSON. Structured Outputs (strict:true) guarantees schema compliance.

Coding & Reasoning Models

These models are specifically optimized for software engineering tasks, including code generation, debugging, and long-horizon development workflows.

GPT-5.3-Codex

- Highly capable **agentic coding model**
- Handles **large codebases and multi-step development tasks**
- Strong debugging and refactoring capabilities
- Suitable for advanced developer tools

GPT-5.1-Codex

- Optimized for **structured programming tasks**
- More efficient for **mid-level coding workflows**
- Good balance between speed and capability
- Useful for automation and backend development

Image Generation Models

These models are designed for generating and editing images from text prompts, enabling creative and design-focused applications.

GPT Image 1.5

- State-of-the-art **image generation quality**
- Produces highly detailed and realistic visuals
- Supports advanced prompt control
- Suitable for professional design and media

GPT Image 1

- Earlier generation image model
- Faster and more cost-efficient
- Good for basic image generation tasks
- Suitable for lightweight applications

Example - Chatbot Application

Model Used: GPT-4-o-mini

Use Case:

- Conversational assistant · customer support · FAQs

```
response = client.chat.completions.create(
  model="gpt-4o-mini",
  messages=[
        {"role": "system",} "content": "You are a helpful chatbot."},
        {"role": "user", "content": "What is AI?"}
  ]
```

Output:

- "AI is the simulation of human intelligence by machines."

Key Points:

- Multi-turn conversation
- Context-aware responses
- Real-time interaction

Example - Image Generation

Model Used: GPT Image 1.5

Use Case:

- Generate images from text prompts
- Design · creative content

```
response = client.chat.completions.create(
  model="gpt-image-1.5",
 prompt="A futuristic city at sunset"
```

Key Points:

- High-quality image generation
- Creative control via prompts
- Useful for design workflows

Output:



Question

- What is the main purpose of roles in OpenAI API?
  - Provide structure to the prompt engineering process.
  - Allow system level instructions
  - Allow structured inputs to be fed into the model
  - Historical legacy



Thank you

Instructor



Sourangshu Bhattacharya
sourangshu@cse.iitkgp.ac.in

Teaching Assistants (IITKGP)



Suman Bera



Saptarshi Mondal